Assigned: 11 May 2026 **Due: 24 May 2026**	**MAS Assignment** **HW No.2**

Multi-Agent Smart Home Request Handling

1. Introduction

In ambient intelligence (AmI) environments, devices and systems must interpret high-level user requests and execute them in the context of current environmental state and user-expressed constraints. A challenge arises when **multiple agents** must collaborate: one agent needs to understand what the user wants, while another manages the environment and enforces user preferences.

This homework explores **deterministic environment management and LLM-supported reasoning** in a practical multi-agent scenario: a RequestSolver agent must coordinate with an EnvironmentManager agent to: 1. **Discover capabilities** — what devices exist and what actions they support 2. **Query state** — what are current property values 3. **Apply constraints** — respect user preferences that block certain device-room-time combinations

The scenario uses a **SmartHome simulator**, where requests involve commanding lights, thermostats, fans, curtains, blinds, and other household devices. Requests can be **single commands** (e.g., “turn on the bedroom light”) or **multi-step** (e.g., “lower the temperature and open the blinds in the living room”). Requests can also refer to **infeasible commands** (e.g. an explicit request to turn on the air conditioning in a room that does not have such a device).

The scenario also includes **user preferences as explicit constraints**. Unlike infeasibility due to missing devices, a user might prefer **not to use a device at a specific time** (e.g., “don’t adjust the fan in the morning — I’m sleeping”). When such a preference is active, a request targeting that device+room+time should fail gracefully.

Main tasks

- Design and implement **deterministic agent interfaces** for environment management
- Learn how to work with **structured representations of information** through a **knowledge graph of the environment** (RDF-based home descriptions and device capabilities)
- Implement multiple strategies for **LLM-supported request reasoning** (full context, sequential exploration, semantic classification)
- Compare performance of different architectural approaches

2. Scenario Setup & Simulator

The SmartHome Environment

Your test dataset contains **36 requests** spanning multiple smart homes. Each request is **grounded in reality**: - Requests reference actual rooms and devices in simulated homes - Expected actions and property values are pre-computed and verified - A **complete SmartHome simulator** is included (see `smart_home_simulator.py`)

Running the Simulator

The simulator runs as a FastAPI service on `http://localhost:8080`. Start it before running the evaluation, using the start_simulator.sh shell script.

This will start the simulator with all 100 smart homes loaded from `simulator_data/`. The simulator serves:

- RDF/Turtle descriptions of home workspaces and artifacts
- Property read endpoints (HTTP GET)
- Action invocation endpoints (HTTP POST) - State reset endpoints

All required home description files (Turtle RDF + initial state JSON) are included in the `simulator_data/` directory.

Device Types

The simulator supports 15 device types:

Device Type	Typical Actions	Key Properties
Light	turn_on, turn_off, set_brightness, set_color	brightness, color, state
Air Conditioner	turn_on, turn_off, set_temperature, set_mode, set_fan_speed	temperature, mode, fan_speed, swing
Heating	turn_on, turn_off, set_temperature, set_mode, set_fan_speed	temperature, mode, fan_speed
Fan	turn_on, turn_off, set_speed, set_swing	speed, swing, state
Blinds / Curtains	open, close, set_degree	degree, state
Humidifier	turn_on, turn_off, set_intensity, set_mode	intensity, mode, tank

Device Type	Typical Actions	Key Properties
Dehumidifier	turn_on, turn_off, set_intensity, set_mode	intensity, mode
Air Purifier	turn_on, turn_off, set_fan_speed, set_mode	fan_speed, mode
Media Player	play, pause, stop, set_volume, set_artist	volume, state, artist
Aromatherapy	turn_on, turn_off, set_intensity, set_interval	intensity, interval
Water Heater	turn_on, turn_off, set_temperature, set_mode	temperature, mode
Garage Door	open, close	state
Vacuum Robot	start, stop, return_to_dock	battery, state
Trash	pack (empty)	state
Other	...	...

Home Layout Example

A typical home has 12–13 rooms (master_bedroom, guest_bedroom, living_room, dining_room, study_room, kitchen, bathroom, foyer, corridor, balcony, garage, store_room) plus home-level artifacts like the vacuum robot.

Example request:


```
{
  "id": "home71_multi_45",
  "issued_at": "09:15",
  "input": "Set the air conditioner in the guest bedroom to 24 degrees and set the
brightness of the light in the study room to 60.",
  "output": [
    {
      "execution": "success",
      "affordance":"http://localhost:8080/workspaces/home71/guest_bedroom/artifacts/
guestBedroomAirConditioner/set_temperature",
      "params": {"temperature": 24},
      "test": {"property": "..../properties/temperature", "expected_value": 24}
    },
    {
      "execution": "success",
      "affordance":"http://localhost:8080/workspaces/home71/study_room/artifacts/
studyRoomLight/set_brightness",
      "params": {"brightness": 60},
      "test": {"property": "..../properties/brightness", "expected_value": 60}
    }
  ]
}
```

Request Types in the Test Set

- **Single Feasible (12 requests)** — one command, device exists, should succeed
- **Single Infeasible (12 requests)** — one command, device absent in that room, must return error
- **Multi Feasible (12 requests)** — 2–3 commands, all feasible
- **Preference-Affected (3 of above)** — a sub-command is blocked by an active user preference

User Preferences

User preferences are time-dependent constraints:

```
[
  {
    "device_type": "air_conditioner",
    "room": "guest_bedroom",
    "dislike_interval": {"start": "10:00", "end": "13:00"},
    "reason": "avoid AC adjustments during guest nap time"
  }
]
```

Semantics: If a request is issued at a time issued_at that falls within [start, end) of a preference, and the request targets that device_type in that room, that sub-command should be treated as **infeasible** (return execution: "error_input").

3. Assignment Requirements

You must **implement the EnvironmentManager and RequestSolver agents**, then run three **distinct strategies** for request interpretation.

Part A: Implement EnvironmentManager [20p]

Implement all 5 methods in smart_home_hw/environment_manager.py:

1. **get_rooms(home_id)**
    - Query the simulator's workspace description (RDF/Turtle format)
    - Parse HMAS ontology to find sub-workspaces
    - Return room names
2. **get_artifacts_in_room(home_id, room)**
    - Query the simulator's room workspace description
    - Find all artifacts (devices) contained
    - Return artifact URIs or names
3. **get_artifact_affordances(artifact_uri)**
    - Fetch the artifact's Thing Description (WoT TD)
    - Parse to extract:
        - Action affordances (name, URI, input parameter schemas)
        - Property affordances (name, URI for reading)
    - Return ArtifactInfo with actions and properties lists
4. **get_artifact_state(artifact_uri, property_name=None)**
    - If property_name given: fetch only that property's value
    - Else: fetch all properties
    - Make HTTP GET requests to property URIs
    - Return ArtifactState
5. **get_active_preferences(issued_at)**
    - Parse issued_at (format "HH:MM")
    - For each preference, check if time falls in [start, end)
    - Return list of active preferences only

Hints: - Simulator base endpoints: GET /workspaces/{home_id}/... (RDF) - Property URIs in requests follow pattern: .../properties/{property_name} - Use requests library for HTTP calls; rdflib for RDF parsing (optional, or regex)

Part B: Implement RequestSolver with Three Strategies [60p]

Implement `solve(request)` in `smart_home_hw/request_solver.py`. Your implementation should support **three strategies** (use a parameter, subclasses, or a choice variable):

Strategy 1: Full Context [20p]

Approach: Fetch **all** environment information upfront, then pass it to the LLM in a single prompt.

1. Call env_manager.get_rooms(home_id)
2. For each room: get_artifacts_in_room()
3. For each artifact: get_artifact_affordances() + get_artifact_state()
4. Call get_active_preferences(issued_at)
5. Construct a comprehensive prompt with:
   - Request: request.input
   - All homes, rooms, artifacts, affordances, states
   - Active preferences
   - Instruction: "Map each sub-goal to affordances or report error_input"
6. Call LLM once
7. Parse LLM response to extract action URIs and parameters

Trade-off: Pro—coherent reasoning, single LLM call. Con—high token count, wasteful for simple requests.

Strategy 2: Sequential Affordance Exploration [20p]

Approach: Iteratively discover only relevant affordances and state.

1. Call LLM to infer goal keywords and likely device types/rooms from request
2. Based on inferred types, selectively query:
   - get_artifacts_in_room() for relevant rooms
   - get_artifact_affordances() for those artifacts only
3. Call get_artifact_state() only for discovered artifacts
4. Call get_active_preferences(issued_at)
5. Call LLM to map sub-goals to discovered affordances
6. Return ActionOutputs

Trade-off: Pro—efficient, fewer unnecessary queries. Con—multiple LLM calls, risk of missing affordances.

Strategy 3: Semantic Classification + Code Generation [20p]

Approach: Classify each sub-goal as “set” (absolute value) or “modify” (relative change), then generate imperative code.

1. Call LLM to parse request and classify each sub-goal:
   - Sub-goal type: "set_property" (absolute) vs "adjust_property" (relative)
   - Target: room, device type, property name
   - Value: new value (for set) or delta (for adjust)
2. For each classified sub-goal:
   a. Discover relevant artifact (get_rooms, get_artifacts_in_room, get_affordances)
   b. Compute final value (for adjust, fetch current state, add delta)
   c. Find matching action affordance
   d. Generate ActionOutput
3. Check active_preferences; if blocked, return error_input

Trade-off: Pro—deterministic, testable value computation. Con—requires careful parsing, multi-step per sub-goal.

Part C: Run Evaluation & Compare Strategies [20p]

```
cd smart_home_hw
python run_evaluation.py
```

Evaluation Flow

The evaluation engine verifies your agent's predictions as follows:

For each request:

1. Agent predicts: list[ActionOutput] with affordance + params

2. Evaluation engine:
   For each predicted ActionOutput:
   a. If execution="error_input": verify it matches ground truth (expected error)
   b. If execution="success":
   - Execute the action at the predicted affordance with predicted params
   - Fetch the property value(s) from the simulator
   - Compare against ground truth's expected value
   - Record success/failure

   c. Reset simulator state (reset home to initial state)

3. Compute metrics across all requests:
   - success_rate: % of correctly predicted actions
   - property_accuracy: % of property values that matched expected

- action_f1: harmonic mean of action precision & recall
- impossible_detection_rate: % of error_input predictions that were correct

Your program should:

1. Load ../requests.json (36 requests) and ../preferences.json (3 preferences)
2. For each request:
   a) Extract home_id from request.id
   b) Call solver.solve(request) using your chosen strategy
   c) Evaluate against ground truth (success_rate, action_f1, property_accuracy, impossible_detection_rate)
3. Store and print aggregated metrics in a table format

Deliverables:

You must submit two files:

1. **An archive** with your code that implements the agents and the evaluation of their reasoning strategies. The file follows the naming convention: **`MAS-HW2-<first_name>-<last_name>-code.zip`**
2. **A PDF Report (MAS-HW2-<first_name>-<last_name>-report.pdf)** with:
   - Brief description of each strategy implementation
   - Metrics table for each strategy (success_rate, F1, property_accuracy, impossible_detection_rate)
   - Comparison and analysis: which strategy performs best? Why?
   - Lessons learned about LLM-supported reasoning and communication in agent applications

4. Tips

1. **Start with EnvironmentManager**: It's prerequisite for everything else. Test each method independently.
2. **HTTP & RDF**: Simulator returns RDF/Turtle. You can parse with rdflib or regex + string processing.
3. **LLM Prompting**: For Strategy 1, be explicit: list all affordances, ask LLM to match sub-goals to actions.
4. **Testing**: Write a small script to test one request end-to-end before running full evaluation.
5. **Error Handling**: Some property URIs might return 404 (device absent). Handle gracefully in evaluation.

Appendix: API Reference

Simulator Endpoints

```
# Fetch home workspace (RDF Turtle)
GET /workspaces/{home_id}

# Fetch room workspace (RDF Turtle)
GET /workspaces/{home_id}/{room}

# Fetch artifact Thing Description
GET /workspaces/{home_id}/{room}/artifacts/{artifact_name}

# Read a property
GET /workspaces/{home_id}/{room}/artifacts/{artifact_name}/properties/
{property_name}
Response: {"value": <actual_value>}

# Invoke an action
POST /workspaces/{home_id}/{room}/artifacts/{artifact_name}/{action_name}
Body: {"param1": value1, ...}
Response: {"status": "success", "message": "..."}

# Reset home state
POST /reset
Body: {"home": "home71"}
```

Model Schemas

Request (input):

```
@dataclass
class Request:
    id: str                     # e.g., "home71_one_101"
    issued_at: str              # e.g., "09:15"
    input: str                  # Natural language
    output: list[ActionOutput]  # Ground truth
```

ActionOutput (expected or predicted):

```
@dataclass
class ActionOutput:
 execution: str           # "success" or "error_input"
 affordance: str | None   # Action URI if success
 params: dict             # Action parameters
 test: dict               # {"property": uri, "expected_value": value}
```

Preference (active constraints):

```
@dataclass
class Preference:
 device_type: str               # "light", "fan", etc.
 room: str                      # "master_bedroom", etc.
 dislike_interval: TimeInterval # {"start": "HH:MM", "end": "HH:MM"}
 reason: str
```

Evaluation Metrics

```
class EvaluationMetrics:
    success_rate               # successful_tests / total
    quantifiable_rate          # partially_successful / total
    action_f1                  # Harmonic mean of precision & recall
    property_accuracy          # matched_properties / checked_properties
    impossible_detection_rate  # detected_error_inputs/expected_error_inputs
```